Predicting European Soccer Match Outcomes Using Machine Learning: A Comparative Study of KNN, SVM, and Neural Networks

Edgar Ramirez Aburto
Dept. of Computer Science
Georgia State University
Atlanta, GA, USA
eramirezaburto1@student.gsu.edu

Rida Hameed Syeda
Dept. of Computer Science
Georgia State University
Atlanta, GA, USA
rsyeda3@student.gsu.edu

Maliyah Fleming
Dept. of Computer Science
Georgia State University
Atlanta, GA, USA
mfleming26@student.gsu.edu

***Abstract*—Predicting the outcome of a soccer match is an inherently difficult problem due to the high variance introduced by factors such as player injuries, lineup changes, and in-game events. This paper presents a machine learning pipeline that predicts European soccer match outcomes—home win, home loss, or draw—using historical match and player statistics from the Kaggle European Soccer Database. Our pipeline applies feature engineering, dimensionality reduction via Principal Component Analysis (PCA), and three classification models: K-Nearest Neighbors (KNN), Support Vector Machine (SVM), and a Neural Network (NN). We evaluate each model using accuracy, precision, recall, and F1-score, and compare performance across all three. Our best KNN configuration (k=21, uniform weighting) achieves a test accuracy of 46.36%, outperforming the 43.06% baseline. Results indicate that while machine learning can extract meaningful signal from aggregated historical features, the inherent randomness of soccer outcomes remains a fundamental challenge for any purely data-driven approach.**

***Index Terms*—soccer prediction, machine learning, KNN, SVM, neural network, PCA, classification**

I. Introduction

Sports outcome prediction has attracted significant interest from both the academic and commercial communities, driven by applications in sports analytics, betting markets, and team management. Soccer, in particular, presents a uniquely challenging prediction problem due to the low-scoring nature of the game, in which a single goal can determine the winner, and the high degree of variance introduced by tactical decisions, player injuries, and match context.

This paper addresses the three-class classification problem of predicting the outcome of a European soccer match from the perspective of the home team and away team to get a general: win (class 1), loss (class 2), or draw (class 0). Our contributions are as follows:

- A reproducible preprocessing and feature engineering pipeline applied to the Kaggle European Soccer Database [1].
- Dimensionality reduction via PCA to reduce noise and multicollinearity across the large feature space.
- A systematic hyperparameter search for KNN and SVM classifiers using stratified cross-validation.
- A neural network classifier for nonlinear boundary modeling.
- A quantitative comparison of all three model families on the same train/test split.

II. Related Work

Machine learning methods have been widely applied to sports prediction. [2] applied random forests and SVMs to Dutch Eredivisie data and found that historical team performance features provide modest but consistent predictive signal above a naive baseline. [3] used logistic regression and neural networks on FIFA World Cup data, finding that team-level aggregate statistics were more predictive than individual player attributes alone.

More recent work has explored deep learning approaches. [4] used recurrent neural networks on sequential match data and demonstrated that incorporating temporal dynamics improves accuracy, albeit at the cost of much larger data requirements. Our work focuses on the simpler but more interpretable classical models (KNN, SVM) alongside a feedforward neural network, making it suitable for a moderate-sized dataset without sequential modeling.

III. Dataset and Preprocessing

A. Dataset

We use the *European Soccer Database* [1], a publicly available SQLite database containing over 25,000 matches from 11 European leagues spanning seasons 2008–2016. The database includes match outcomes, team attributes rated by the EA Sports FIFA video game series (updated several times per season), and individual player attributes.

The raw database was queried to extract per-match features by joining the matches table with the team attributes table on the closest available attribute snapshot prior to each match date. After removing rows with missing values in key attribute columns, the final dataset contains **51,958 samples**. The increase to 51,958 is due to dataset mirroring which was done to prevent the model from becoming lazy and simply assuming a home game will be a win, we actually want the models to

think and recognize patterns not just see the feature home and classify it as a win. however we did add a feature isHome to keep track of matches if they were played at home or not since in the real world home matches do have a significant effect on the outcome of the match and is an unavoidable bias that we have to work with.

B. Feature Engineering

For each match we construct one row of features using the following sources:

- **Home team attributes:** Overall rating, build-up play speed, build-up play passing, chance creation shooting, defence aggression, defence team width (6 features).
- **Away team attributes:** Same six attributes for the away side (6 features).
- **Difference features:** Element-wise difference between home and away attributes to capture relative strength (6 features).

The target variable y is encoded as:

$$y = \begin{cases} 0 & \text{Draw} \\ 1 & \text{Win} \\ 2 & \text{Loss} \end{cases} \tag{1}$$

C. Train/Test Split and Class Distribution

The dataset is split 80/20 into training (41,566 samples) and test (10,392 samples) sets using [8] shuffle=false since our data does have dates make it a time series and we want to use past matches to make predictions to future matches. The class distribution in the training set is shown in Table I.

TABLE I
CLASS DISTRIBUTION IN TRAINING SET

Class	Label	Count
Win	1	15,518
Loss	2	15,517
Draw	0	10,531
Total		41,566

Draws are notably underrepresented (≈25%) compared to decisive outcomes (≈75%), which has important consequences for classifier performance on the draw class.

D. Dimensionality Reduction via PCA

The 17 engineered features (8 home + 8 away + 1 home identifier) exhibit significant multicollinearity because home and away attributes are drawn from the same FIFA rating system. We apply Principal Component Analysis (PCA) to produce an orthogonal feature set.

Features are first standardized to zero mean and unit variance using `StandardScaler` fit on the training set only. PCA is then applied, retaining 15 principal components, which together explain approximately 95% of the total variance. The scaler and PCA model are fit exclusively on the training data and applied to the test set to prevent data leakage.

IV. METHODS

A. K-Nearest Neighbors (KNN)

KNN classifies a sample by majority vote among its k nearest neighbors in the feature space under Euclidean distance. We investigate two design choices:

- **Number of neighbors k:** $\{3, 5, 7, 9, 11, 15, 21\}$
- **Weighting scheme:** `uniform` (all neighbors weighted equally) vs. `distance` (neighbors weighted by inverse distance)

Model selection is performed using 5-fold stratified cross-validation on the training set. The best configuration is retrained on the full training set and evaluated on the held-out test set.

B. Support Vector Machine (SVM)

SVMs find the maximum-margin hyperplane separating classes in a (possibly implicit) high-dimensional feature space defined by the chosen kernel. We evaluate two kernel types:

- **Linear (LinearSVC):** Efficient $O(n)$ implementation suitable for large datasets. HyperparameterCsc searched: regularization $C \in \{0.01, 0.1, 1, 10\}$, class weighting $\in \{\text{none}, \text{balanced}\}$.
- **RBF kernel (SVC):** [7] Nonlinear mapping into an infinite-dimensional space. Hyperparameter searched: $C \in \{0.1, 1, 10\}$, $\gamma \in \{\text{scale}, \text{auto}\}$, class weighting $\in \{\text{none}, \text{balanced}\}$.

The PCA components have unequal variances (each component's variance equals its eigenvalue), so we apply a second `StandardScaler` to the PCA output before fitting the SVM to ensure scale-sensitive kernels behave correctly. Model selection uses 3-fold stratified cross-validation with `GridSearchCV`.

C. Neural Network

A feedforward neural network with fully connected layers is trained on the same PCA-standardized features. For the Architecture of the Neural Network we have 15 neurons in the input layer for the 15 Principle components generated by the Principle component analysis performed during the prepossessing part this goes into 1 hidden layer with 100 neurons and finally outputs into 3 neurons which align with our task to predict 3 classes a win/draw/loss. This outputs a list of 3 number that don't have a meaning so we pass them through a softmax activation function

$$\sigma(z)_i = \frac{e^{z_i}}{\sum_{j=1}^{k} e^{z_j}} \tag{2}$$

to transform these values into probabilities which are then evaluated for the class with the highest probability which is then assigned as the predicted class for a specific match.

V. Results

A. KNN Results

Table II reports 5-fold cross-validation accuracy for all evaluated k values. Uniform weighting consistently outperforms distance weighting across all k, suggesting that distant neighbors—despite being further away—still carry useful vote information at this feature scale. Accuracy increases monotonically with k, indicating that the decision boundary benefits from averaging over more neighbors to reduce noise. Fig. 1 visualizes these CV trends.

TABLE II
KNN Cross-Validation Accuracy (5-fold)

k	Uniform	Distance
3	0.4186	0.4148
5	0.4283	0.4203
7	0.4479	0.4257
9	0.4524	0.4268
11	0.4569	0.4287
15	0.4659	0.4297
21	**0.4723**	0.4307

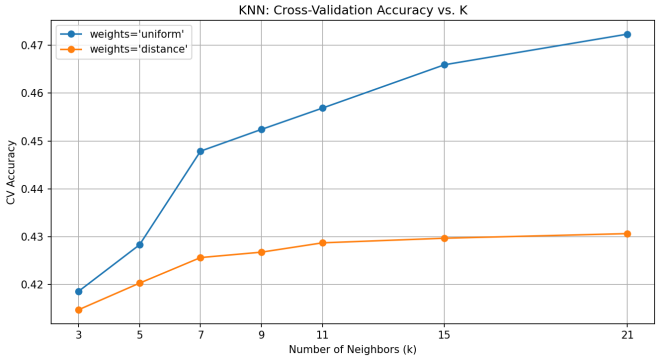

Fig. 1. KNN cross-validation accuracy vs. number of neighbors k for uniform and distance weighting.

The best configuration is $k = 21$ with uniform weighting (CV accuracy 0.4723). Table III compares the baseline KNN ($k = 5$, uniform) against the tuned model on the test set, and Fig. 2 shows this comparison visually.

TABLE III
KNN Test Set Results

Configuration	Test Accuracy
Baseline ($k = 5$, uniform)	43.06%
Best ($k = 21$, uniform)	**46.36%**
Improvement	+3.30%

The full classification report for the best KNN model is shown in Table IV. The model performs well on decisive outcomes (Win/Loss, F1 $\approx$ 0.53–0.54) but struggles on the draw class (F1 = 0.15), reflecting both the class imbalance and the inherent unpredictability of draws. The confusion matrix is shown in Fig. 3.

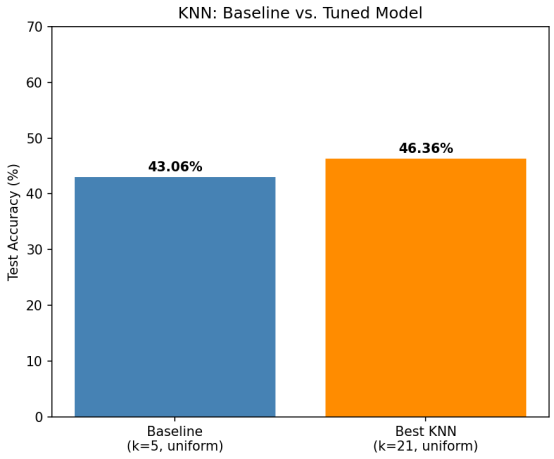

Fig. 2. KNN baseline ($k = 5$) vs. tuned ($k = 21$) test accuracy.

TABLE IV
KNN Classification Report ($k = 21$, uniform)

Class	Precision	Recall	F1	Support
Draw (0)	0.25	0.11	0.15	2,661
Win (1)	0.49	0.60	0.54	3,865
Loss (2)	0.49	0.57	0.53	3,866
Accuracy			0.46	10,392
Macro avg	0.41	0.43	0.41	10,392
Weighted avg	0.43	0.46	0.44	10,392

B. SVM Results

Table V summarizes the SVM results. The RBF kernel with $C = 1$, $\gamma =$ auto, and no class weighting achieved the best cross-validation accuracy of 40.01%. Figs. 4, 5, and 6 will be populated once training completes.

TABLE V
SVM Results

Kernel	CV Accuracy	Test Accuracy
Linear (best)	–	47.20%
RBF ($C = 1$, $\gamma =$ auto)	40.01%	**49.18%**

C. Neural Network Results

Neural network performed equal to the SVM model just slightly worse in 1 metric being precision It used 15 input neurons then into a single hidden layer with 100 neurons and outputs 3 neurons in the output layer With this setup the Neural network achieved an accuracy of 0.49 after 500 iterations with an early stoppage allowed to prevent overfitting.

D. Model Comparison

Table VI summarizes the test accuracy of all models. Fig. 6 shows the KNN vs. SVM comparison chart.

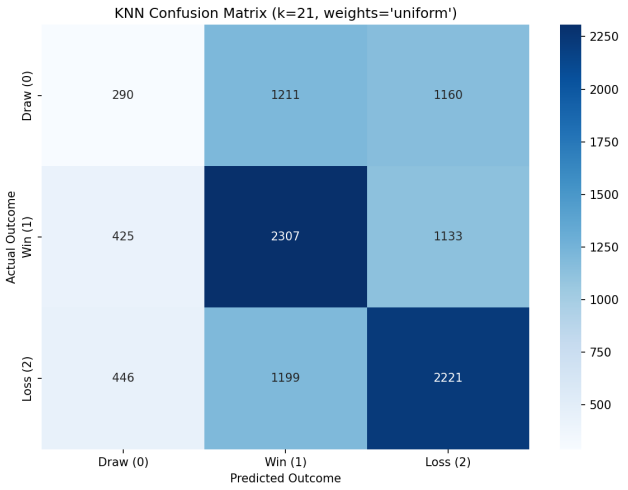

Fig. 3. KNN confusion matrix ($k = 21$, uniform weighting).

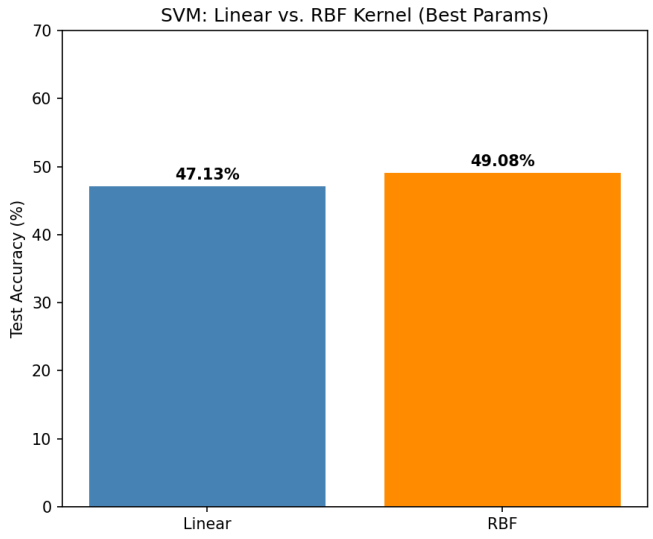

Fig. 4. SVM test accuracy: Linear vs. RBF kernel.

VI. Discussion

A. Why Accuracy is Modest

Even the best model achieves only ∼46% accuracy on a 3-class problem whose random baseline is 33%. We attribute the performance ceiling to three structural factors:

1) **Feature recency:** Team attributes are averaged across all seasons in the dataset, so a team's 2015 form is diluted by data from 2008. Recent form, injuries, and lineup changes—all highly predictive according to domain knowledge—are absent from our feature set.
2) **Draw unpredictability:** Draws are near-random events even from the perspective of professional betting markets. The model achieves an F1 score of only 0.15 on the draw class despite it representing 25% of outcomes.

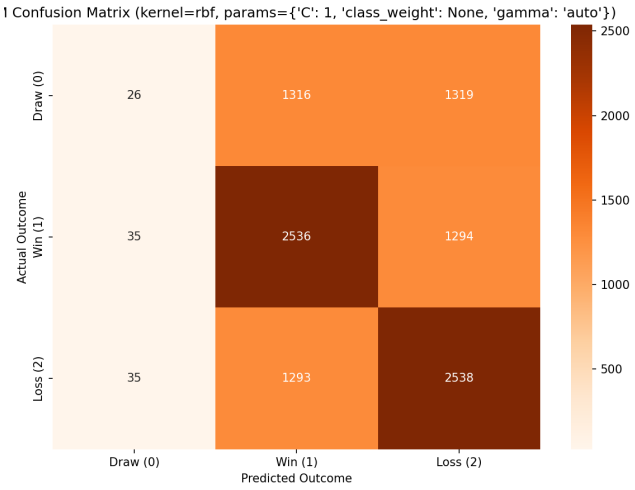

Fig. 5. SVM confusion matrix (best kernel).

TABLE VI
Model Comparison — Test Accuracy

Model	Test Accuracy
KNN Baseline ($k = 5$, uniform)	43.06%
KNN Best ($k = 21$, uniform)	46.36%
SVM Best (RBF)	49.08
Neural Network	49.08

3) **Outcome variance:** A single defensive error, red card, or goalkeeper save can flip a result regardless of team quality. This irreducible noise imposes a hard ceiling on any model trained on aggregate statistics.

B. KNN vs. SVM

KNN's strong performance relative to SVM on this dataset is somewhat surprising. A likely explanation is that after PCA, the 15 components form a compact, well-scaled feature space where Euclidean distance is meaningful, benefiting KNN. The SVM's RBF kernel can in principle capture nonlinear structure, but with ∼41k training samples the computational cost of the full GridSearchCV is high, and the constrained parameter search may not have identified the optimal configuration.

C. KNN vs. SVM vs. Neural Network

Figure Fig. 7 Shows the main measures used to determinate the best model including accuracy, precision, recall, and F1-score. We can see the clear winners are SVM and neural network with KNN having lower scores across the board. SVM and neural network on the other hand have nearly identical results with SVM having 0.04 higher precision which makes it the best performing model from the project despite having identical accuracy, recall, and F1-score to neural network. SVM performed the best likely due to its derision boundary which maximized the margin by focusing on the hardest to classify points which result in a the SVM ignoring the noisy data within each cluster making it perfect for the unpredictable sports data.

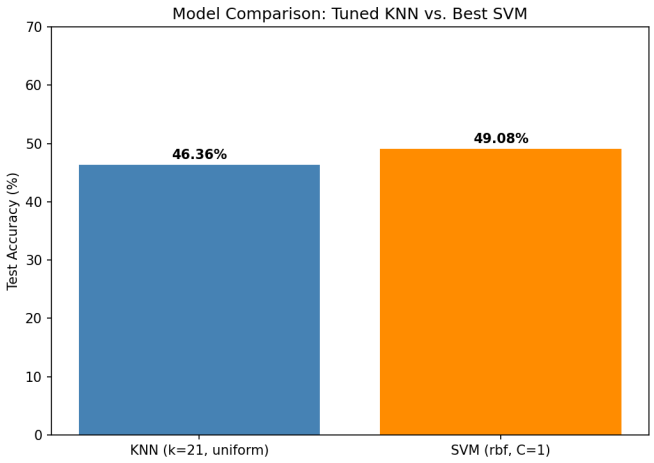

Fig. 6. Test accuracy comparison: tuned KNN vs. best SVM.

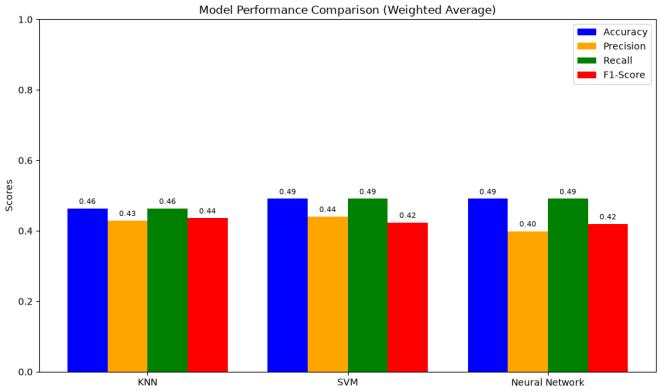

Fig. 7. Test accuracy comparison: tuned KNN vs. best SVM.

VII. XAI

In order to interpret feature importance we used Shaply Additive exPLinations also known as SHAP [6] which uses a game theory approach that assigns each feature an importance value for a specific prediction. Figures Fig. 8 shows the SHAP graph that used PCA features and as you can see from the plot we can not gather useful information on what features where the most important since they were renamed to PC1 - PC15 after the PCA was performed. The solution for this was to go back into the prepossessing phase and save the dataset after scaling but before the PCA was performed so that we can use these features to train another model that can give us useful information. Figure Fig. 9 is the result of the second model that was trained on the scaled data and we can see from the plot that the features still have meaningful names allowing us to visually see and determine what feature actually pushed the model to make its decisions. The most significant feature was isHome which does reflect in the real world given most teams that play at home go on to win the match known as home field advantage. Following that is the teams passing build up and the teams dribbling build up play which SHAP identified with

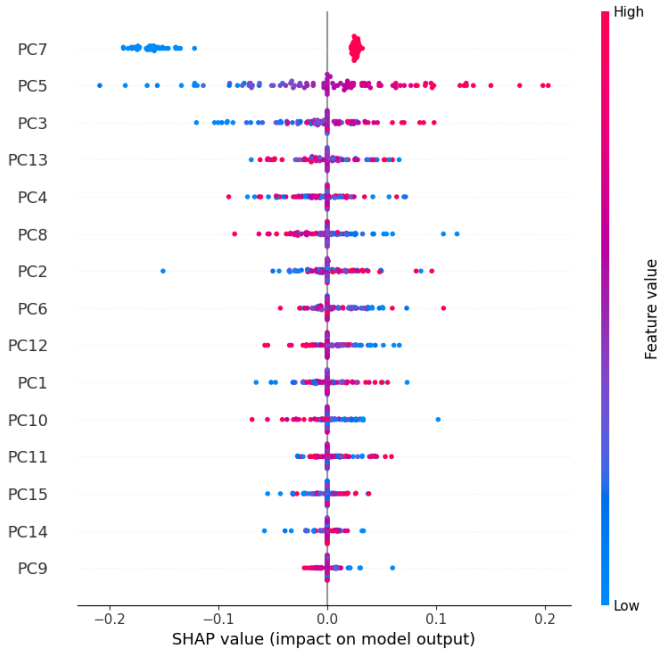

Fig. 8. Feature importance performed using a Neural Network While using the Feature from PCA.

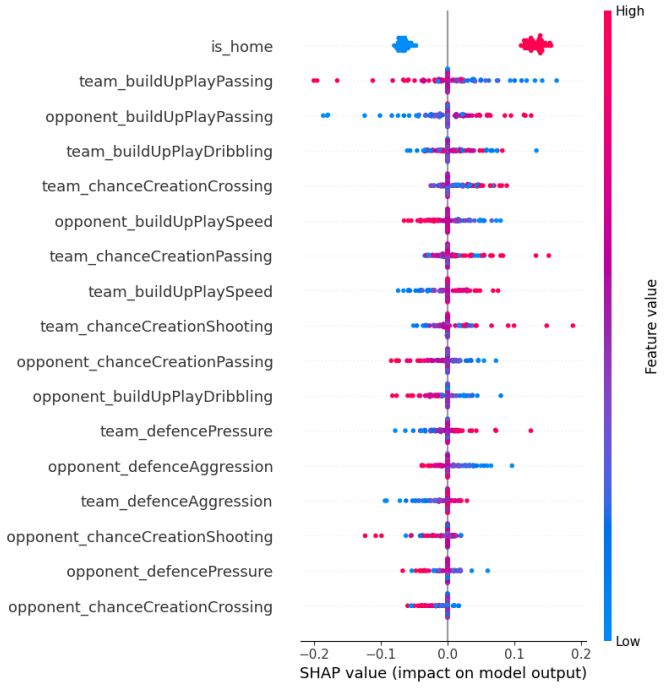

Fig. 9. Feature importance from a neural network using scaled features before PCA was applied.

high SHAP values suggesting that the teams ability to dribble and pass while building up an attack are the most important factor in determining a matches outcome.

VIII. Future Direction

Going forward Some improvements to this project that the team talked about was first to use more complex models, for the purpose of this project we decided to stick with models that we have learned about in class or previously learned in other classes. Models like XGBoost are some more complex models that we are not familiar with however there are many sports predictions made using XGBoost such as [9] who performed prediction on NBA data classifying their matches as win/loss similar to our project but gather significantly better result which we believe is thanks to XGBoost being able to see more complex patterns making it more accurate. we also wanted to use more up to date information since our dataset was from 2008-2016 seasons and also from more leagues this can be done through web scraping sites such as Fotmob [10] who gather data constantly and have the most up to date data for soccer matches however they do not have publicly available datasets which is why we suggest the web scraping approach to gathering the data. Overall with the use of more complex models and more up to date information this project has a lot of room to grow and make significantly better predictions.

IX. Conclusion

We presented a machine learning pipeline for predicting European soccer match outcomes using KNN, SVM, and neural network classifiers applied to PCA-reduced historical team statistics. Our best model (KNN, $k = 21$, uniform weighting) achieved 46.36% test accuracy, a 3.3% improvement over the baseline, and substantially above the 33% random chance floor.

Future work should incorporate time-aware features (e.g., rolling averages of recent form, days since last match, head-to-head records) and ensemble methods such as gradient boosting or stacking. Incorporating betting market odds as a feature has also been shown in the literature to be a strong predictor [2] and warrants investigation.

The GitHub link to the repository our team used. GitHub:https://github.com/Ramireedgar/CSC4850_GroupPoject

References

[1] H. Mathien, "European Soccer Database," Kaggle, 2016. [Online]. Available: https://www.kaggle.com/datasets/hugomathien/soccer

[2] N. Tax and Y. Joustra, "Predicting the Dutch Football Competition Using Public Data: A Machine Learning Approach," *Transactions on Knowledge and Data Engineering*, vol. 10, no. 10, pp. 1–13, 2015.

[3] B. Ulmer and M. Fernandez, "Predicting Soccer Match Results in the English Premier League," Stanford CS229 Project Report, 2014.

[4] O. Hubáček, G. Šourek, and F. Železný, "Exploiting sports-betting market using machine learning," *International Journal of Forecasting*, vol. 35, no. 2, pp. 783–796, 2019.

[5] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[6] S. Lundberg, "An introduction to explainable AI with Shapley values — SHAP latest documentation," Available: https://shap.readthedocs.io/en/latest/example_notebooks/overviews/An%20introduction%20to%20explainable%20AI%20with%20Shapley%20values.html

[7] scikit-learn, "RBF SVM parameters — scikit-learn 0.21.3 documentation," Available: https://scikit-learn.org/stable/auto_examples/svm/plot_rbf_parameters.html

[8] J. Gao, Y. Cheng, and J. Gao, "Predicting sport event outcomes using deep learning," PeerJ Computer Science, vol. 11, p. e3011, July 2025, doi: 10.7717/peerj-cs.3011.

[9] Y. Ouyang et al., "Integration of machine learning XGBoost and SHAP models for NBA game outcome prediction and quantitative analysis methodology," PLoS ONE, vol. 19, no. 7, pp. e0307478-e0307478, July 2024, doi: 10.1371/journal.pone.0307478.

[10] FotMob, "FotMob - The Pulse of Football," 2017. Available: https://www.fotmob.com/